Knowledge Organiser — 1.2 Software & Software Development

OCR A Level Computer Science (H446) — Component 01, Section 1.2 (1.2.1 Operating systems · 1.2.2 Applications generation · 1.2.3 Software development · 1.2.4 Types of programming language)

1.2.1 Operating systems

OS = software managing hardware + providing a platform for applications (uses **abstraction** to hide complexity). **Roles:** memory management · CPU scheduling · interrupt handling · I/O & peripherals (drivers) · user interface (GUI/CLI) · security & file management.

Memory management

	Paging	**Segmentation**
Block size	**Fixed** (pages↔frames)	**Variable** (segments)
Divided by	OS / physical	Logical structure of program
Suffers	Internal fragmentation	External fragmentation
Page table	Maps logical page → physical frame	—

- **Virtual memory:** uses **secondary storage (swap/page file) as if it were RAM** → run programs larger than RAM / more open at once. *Slower than real RAM.*
- **Page fault:** needed page is on disk → OS swaps it back in. **Disk thrashing:** excessive swapping → machine grinds to a halt.

Interrupts & ISR

- **Interrupt** = signal (HW/SW) that a device/process needs CPU attention → avoids **polling**. Have **priorities** (priority queue/interrupt register).
- **ISR (interrupt service routine / handler)** = OS code that deals with a specific interrupt.
- **Handling (in order):** 1 CPU checks interrupt register at **end of each F-D-E cycle** → 2 finish current instruction (if interrupt higher priority) → 3 **push registers (PC, ACC) onto stack** → 4 load address of correct **ISR** → 5 run ISR → 6 **pop registers off stack**, resume task. *Nested interrupts work because stack is LIFO.*

Scheduling (illusion of simultaneous processes; max CPU use, fair, good response)

Algorithm	How	Pre-emptive	Notes
Round robin	Fixed **time slice (quantum)** each, in turn; unfinished → back of queue	Yes	Fair, no starvation; ignores priority/length
First come first served (FCFS)	Run in arrival order to completion	No	Simple; long jobs delay short (**convoy effect**)
Shortest job first (SJF)	Shortest **total** runtime next	No	Min avg wait; needs runtime known; long jobs **starve**
Shortest remaining time (SRT)	Shortest **remaining** time next; shorter arrival pre-empts	Yes	Good for short jobs; starvation; switching overhead
Multi-level feedback queue (MLFQ)	Multiple priority queues; jobs move between them by behaviour (CPU-hungry demoted)	Yes	Flexible; complex to tune; approximates SJF

OS types

Type	Key feature	Example
Distributed	Runs across multiple networked computers as one system	Server clusters
Embedded	Built for one device, limited resources, in ROM, efficient/reliable	Washing machine, smart TV
Multitasking	Several programs at once via time-slicing	Windows, macOS, Linux desktop
Multi-user	Several users share resources; scheduler + permissions	Mainframe, cloud server
Real-time (RTOS)	**Guarantees response within a set time/deadline**	Aircraft, pacemaker, ABS

BIOS, drivers, virtual machines

- **BIOS** = firmware on ROM; **first program to run** at start-up → **POST** (Power-On Self-Test checks HW) → initialise HW, store low-level settings (clock, boot order) → **load OS** (bootstrap) into RAM. *BIOS loads the OS; it is NOT the OS.*
- **Device driver** = software letting OS control a specific hardware device; translates OS commands → device instructions. **HW- and OS-specific.**
- **Virtual machine** = software emulating a complete computer. (1) **System VM:** run a whole guest OS inside another (testing/sandboxing/legacy). (2) **Process VM / intermediate code:** source compiled to platform-independent **intermediate code (e.g. Java bytecode)**, run by a VM (e.g. JVM). +Portable "write once run anywhere"; −slower than native, VM uses resources.

1.2.2 Applications generation

- **Application software** = lets user do a real-world task (word processor, browser, game).
- **Utility software** = maintains/optimises the computer itself (defrag, backup, antivirus, compression, firewall). *Antivirus = utility, NOT application.*

	Open source	Closed source (proprietary)
Source code	Available, modifiable	Hidden, not modifiable
Cost	Usually free	Usually paid licence
Support	Community	Official/paid vendor support
Examples	Linux, Firefox, GIMP	Windows, Photoshop

Translators

- **Assembler** → assembly → machine code (≈ one-to-one mnemonic:instruction).
- **Compiler** → high-level → machine code **all at once**, produces standalone **executable** before running.
- **Interpreter** → translates + executes high-level **one line at a time** at run time; no executable.

	Compiler	Interpreter
When	All at once, before run	Line by line, at run time
Output	Standalone executable	None
Program speed	Fast (already machine code)	Slower (translated each run)
Source needed to run?	No (.exe only)	Yes + interpreter
Errors	All reported at end	Stops at first error
Portability	Platform-specific exe	Portable if interpreter exists
Good for	Distributing finished software	Developing/debugging, scripting

Stages of compilation (IN ORDER)

1. **Lexical analysis** — code → **tokens**; remove whitespace/comments; build **symbol table**.
2. **Syntax analysis (parsing)** — tokens checked vs **grammar**; **parse tree** built; **syntax errors** reported here.
3. **Code generation** — produce **object/machine code** from parse tree + symbol table.
4. **(Code) optimisation** — refine code to **run faster / use less memory**, same result.

Linkers, loaders, libraries

- **Library** = pre-written, pre-compiled, tested subroutines reused → saves time, more reliable.
- **Linker** = combines compiled program + library/other modules into one executable, resolving references. **Static** (copies code in, larger, self-contained) vs **Dynamic** (.dll separate, linked at run time, smaller, shared/updatable).
- **Loader** = OS software that copies the executable from storage **into RAM** and prepares it to run.

1.2.3 Software development

Methodologies

Methodology	Key idea	Use when…
Waterfall	Linear/sequential, one stage at a time	Requirements fixed & clear, small project
Agile	Iterative increments + continuous client feedback	Requirements likely to change/unclear
Extreme programming (XP)	Agile + **pair programming** + constant testing (TDD) + customer in team	High code quality essential
Spiral	Iterative + central **risk analysis** each loop	Large, expensive, high-risk project
RAD	Rapid **prototyping** refined by user feedback	Need it fast / vague requirements

Trade-offs: Waterfall simple/documented but inflexible, client sees product only at end. Agile flexible but less docs, cost/time hard to predict. XP very high quality but staff-intensive. Spiral manages risk but expensive/complex. RAD fast but needs skilled devs + committed users, can be less robust.

Lifecycle stages

Analysis (investigate + gather requirements, feasibility) → **Design** (data structures, algorithms, interface, I/O) → **Development/implementation** (code in modules) → **Testing** (vs requirements; normal/boundary/erroneous data; alpha/beta) → **Evaluation** (vs original requirements: effective, usable, robust, maintainable).

1.2.4 Types of programming language

- **Programming paradigm** = style/approach to writing a program. No one language suits every task → pick the paradigm that makes a problem easier to write/maintain/reuse.
- **Procedural** = ordered statements using **sequence, selection, iteration**; organised into **procedures/subroutines**; specifies *how* (Python, C, Pascal).
- **Assembly language** = low-level, uses **mnemonics** (ADD, STA), ≈ one mnemonic:one machine instruction, processor-specific; for speed/size/HW control (drivers, embedded).
- **Little Man Computer (LMC)** = simple model CPU/instruction set to learn assembly & F-D-E.

LMC instructions: `INP` input→ACC · `OUT` output ACC · `STA xx` store ACC→xx · `LDA xx` load xx→ACC · `ADD xx` · `SUB xx` · `BRA xx` jump always · `BRZ xx` branch if ACC=0 · `BRP xx` branch if ACC≥0 · `DAT` data label · `HLT` stop.

Addressing modes (operand `20`; addr 20→35, addr 35→99)

Mode	Operand is…	`LDA 20` loads
Immediate	the actual value	**20**
Direct	the address holding the value	**35**
Indirect	address of a location holding the **address** of the value (pointers)	**99**
Indexed	base address + **index register** contents (arrays)	20+index

OOP concepts

Term	Definition
Class	Template/blueprint defining attributes + methods
Object	A specific **instance** of a class
Instantiation	Creating an object from a class
Attribute/property	A **variable** of a class/object (its data/state)
Method	A **subroutine** of a class (its behaviour)
Encapsulation	Bundle data + methods + **hide data** (private attributes via methods)
Inheritance	Subclass acquires attributes/methods of superclass; can add/override (reuse, DRY)
Polymorphism	Same method name behaves differently per object (overriding)

Class diagram: box split into class name / attributes / methods; `+` public, `-` private; inheritance arrow points **child → parent**.

Must-know definitions

Term	Definition
Operating system	Software managing hardware + providing a platform for applications
Paging	Dividing memory into **fixed-size** blocks (pages/frames)
Segmentation	Dividing memory into **variable-size** logical blocks (segments)
Virtual memory	Using secondary storage as if it were RAM to run more/larger programs
Disk thrashing	Excessive paging between RAM and disk that slows the system
Interrupt	A signal that a device/process needs the processor's attention
ISR	Code run by the OS to deal with a particular interrupt
Round robin	Scheduling giving each process a fixed time slice in turn
Pre-emptive	A running process can be stopped and the CPU reallocated
Real-time OS	Guarantees a response within a set time limit
BIOS	Firmware that runs first, performs POST and loads the OS
Device driver	Software letting the OS control a specific hardware device
Virtual machine	Software emulating a computer / running intermediate code
Intermediate code	Platform-independent code between source and machine code
Utility software	System software that maintains/optimises the computer
Compiler	Translates high-level source into machine code all at once
Interpreter	Translates and executes high-level source one line at a time
Assembler	Translates assembly language into machine code
Linker	Combines compiled program with library/other modules into one executable
Loader	OS software that loads an executable into RAM to run
Programming paradigm	A style/approach to writing programs
Encapsulation	Bundling data + methods together with data hiding
Inheritance	A subclass acquiring attributes/methods of a superclass
Polymorphism	Methods behaving differently depending on the object

Top exam reminders

- **Paging vs segmentation:** the discriminator is **fixed-size (paging)** vs **variable-size/logical (segmentation)** — state it.
- **Virtual memory** does NOT add RAM — it uses **secondary storage** as if it were RAM (slower); mention **thrashing**.
- **Interrupts:** CPU checks at **end of each cycle** and saves registers on a **stack** so the job resumes — never omit the stack.
- **Compiler vs interpreter:** quote *when* translation happens + *whether an executable is produced*; the program runs faster, not the compiling.
- **Compilation order:** lexical (tokens) → syntax (grammar/errors) → code generation → optimisation; don't muddle lexical and syntax.
- **Methodology questions are AO2/AO3** — apply & justify against scenario clues (budget, deadline, clarity of requirements, risk); pure description caps marks.